SESSION DE RATTRAPAGE – JEE1 ET JEE2

Application de facturation

TP2 : Spring Boot + React + MongoDB

Clavstel El Schaddai BOYAOU

Classe : M1 GELO

Plan

- Contexte et objectifs du TP2
- Du TP1 au TP2 : ce qui change
- Architecture et technologies
- Modèle de données MongoDB
- Sécurité : authentification JWT et rôles
- API REST du backend
- Présentation de la plateforme (interfaces)
- Conclusion et perspectives

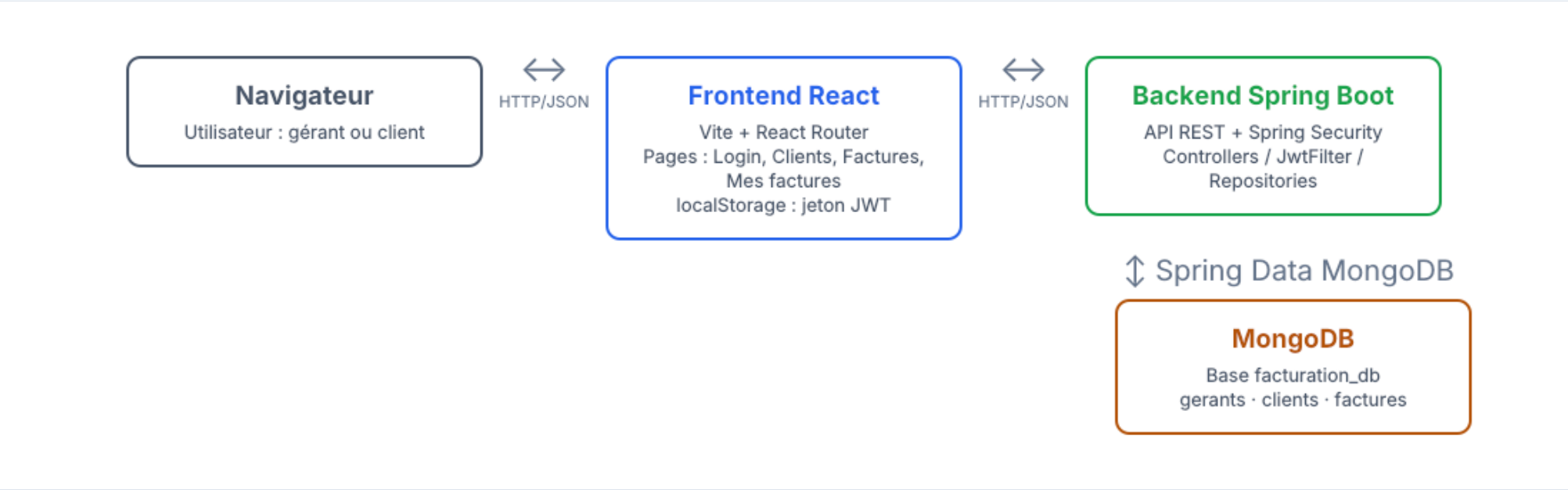
Contexte et objectifs

- **Besoin :** gérer clients et factures d'une entreprise
- **Gérant :** s'authentifie, puis ajoute, modifie, supprime et recherche clients et factures
- **Client (nouveau en TP2) :** se connecte et consulte uniquement ses propres factures
- **Contrainte TP2 :** même architecture que le TP1, mais frontend React et base MongoDB

Du TP1 au TP2

Élément	TP1	TP2
Frontend	HTML + CSS	React (Vite, React Router)
Base de données	MySQL (tables)	MongoDB (collections)
Backend	Spring Boot	Spring Boot (API REST + Spring Data MongoDB)
Authentification	Gérant	Gérant + Client (JWT, rôles)
Échanges	Pages serveur	JSON via HTTP (fetch)

Architecture de la plateforme

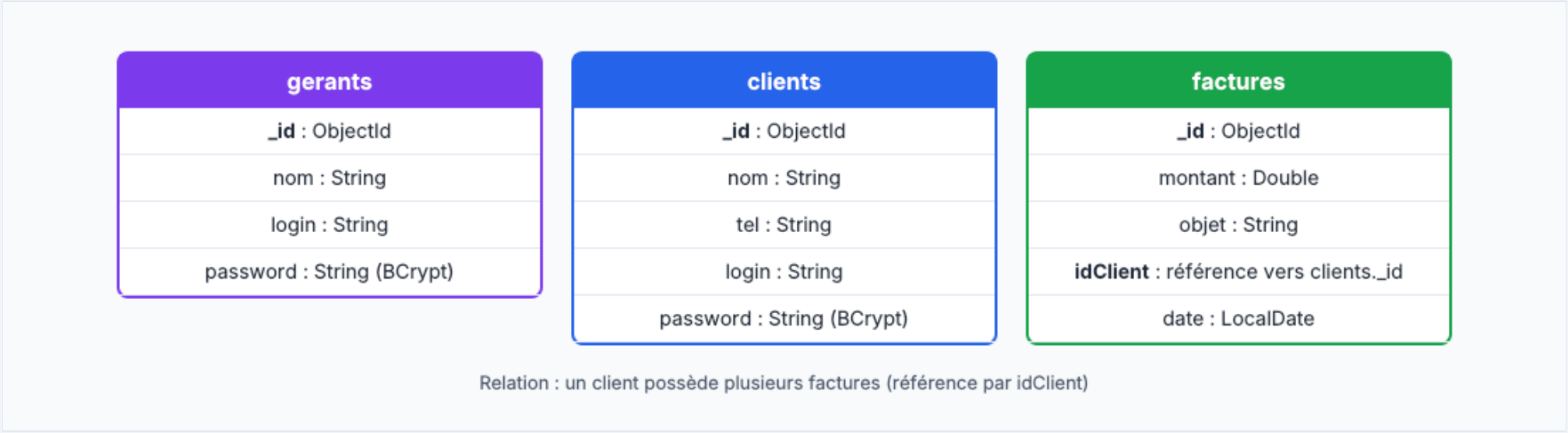


- Frontend et backend séparés, reliés par une API REST (JSON)
- Le backend est sans état : chaque requête porte son jeton JWT

Technologies utilisées

- **Backend :** Java 17, Spring Boot 3.2, Spring Web, Spring Data MongoDB, Spring Security, Bean Validation
- **Sécurité :** JWT (jjwt) + mots de passe chiffrés avec BCrypt
- **Frontend :** React 18, React Router 6, Vite, CSS
- **Base de données :** MongoDB (base facturation_db)
- **Outils :** Maven, npm

Modèle de données MongoDB



Trois collections ; la facture référence son client par idClient

Sécurité : JWT et rôles

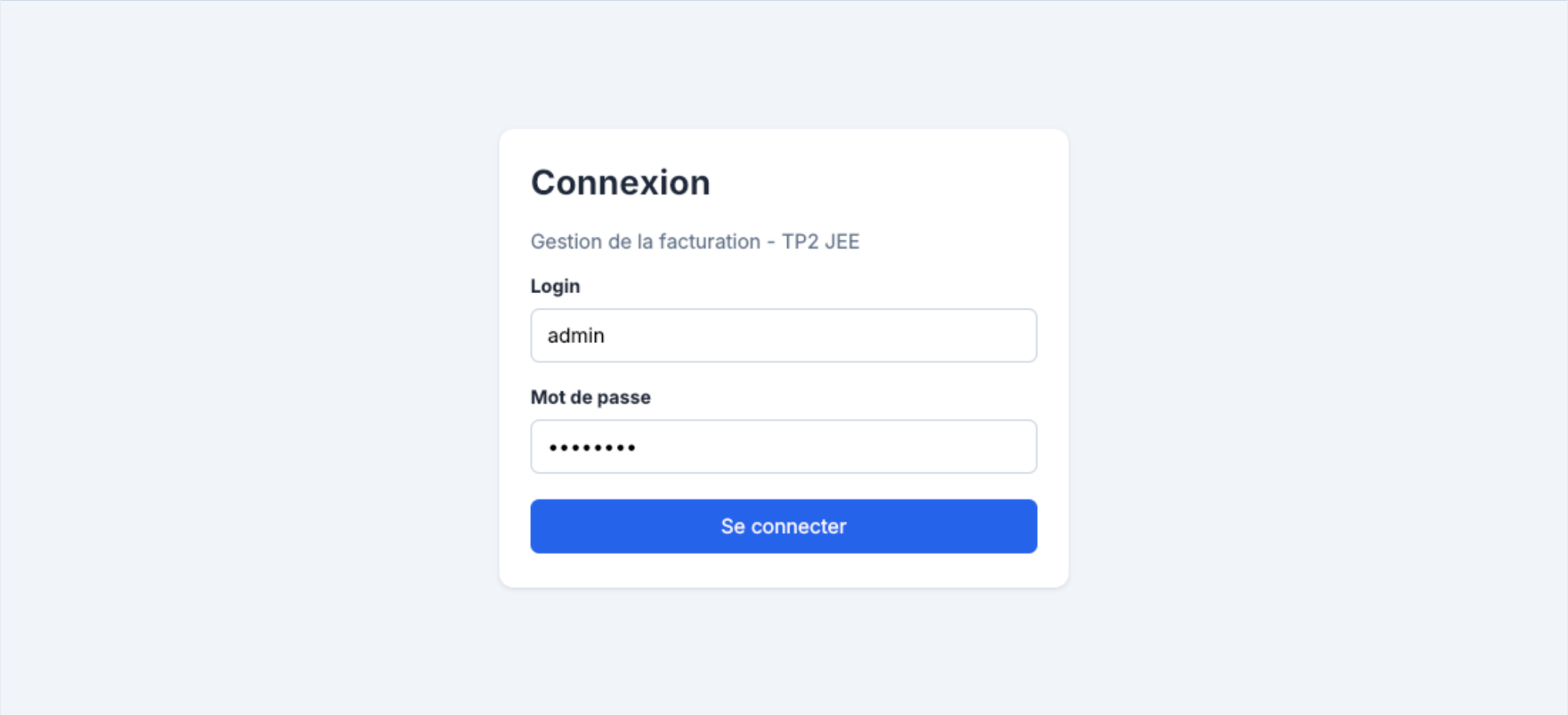
- 1. L'utilisateur envoie login + mot de passe à **POST /api/auth/login**
- 2. Le serveur vérifie le hash BCrypt (gérant d'abord, puis client)
- 3. Il renvoie un **jeton JWT** signé contenant le rôle et l'identifiant
- 4. React stocke le jeton et l'envoie dans l'en-tête **Authorization: Bearer**
- 5. **JwtFilter** valide le jeton ; Spring Security applique les règles par rôle
- **GERANT** : /api/clients et /api/factures | **CLIENT** : /api/factures/mes-factures uniquement
- Le client ne peut lire que ses factures : son id vient du jeton, pas de la requête

API REST

Méthode	URL	Rôle	Description
POST	/api/auth/login	Public	Connexion, renvoie le JWT
GET	/api/clients?q=	Gérant	Lister / rechercher des clients
POST	/api/clients	Gérant	Ajouter un client
PUT	/api/clients/{id}	Gérant	Modifier un client
DELETE	/api/clients/{id}	Gérant	Supprimer (et ses factures)
GET	/api/factures?q=	Gérant	Lister / rechercher des factures
POST / PUT	/api/factures[/{id}]	Gérant	Ajouter / modifier une facture
DELETE	/api/factures/{id}	Gérant	Supprimer une facture
GET	/api/factures/mes-factures	Client	Consulter ses propres factures

Plateforme : page de connexion

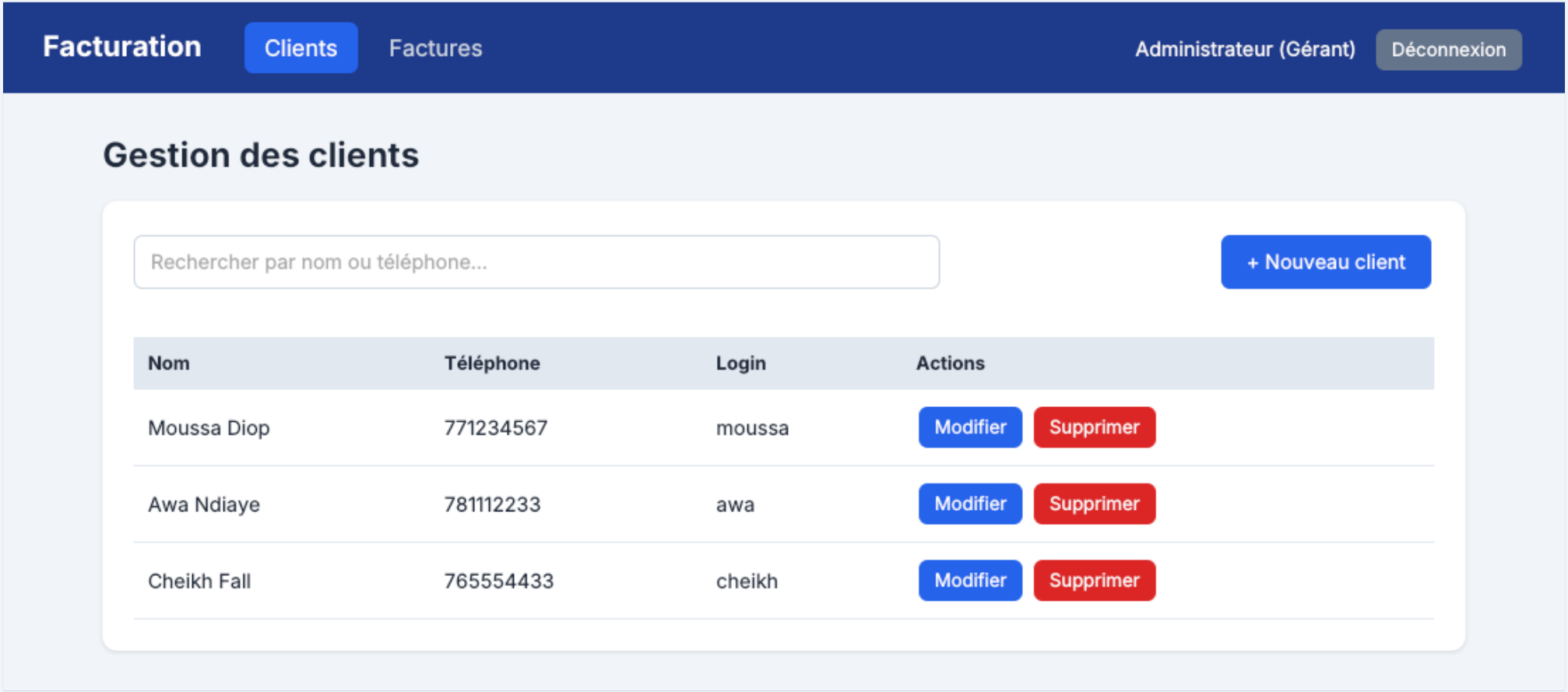
- Un seul formulaire ; le rôle est détecté par le serveur
- Message d'erreur clair si les identifiants sont incorrects



Écran de connexion commun au gérant et au client

Espace gérant : clients

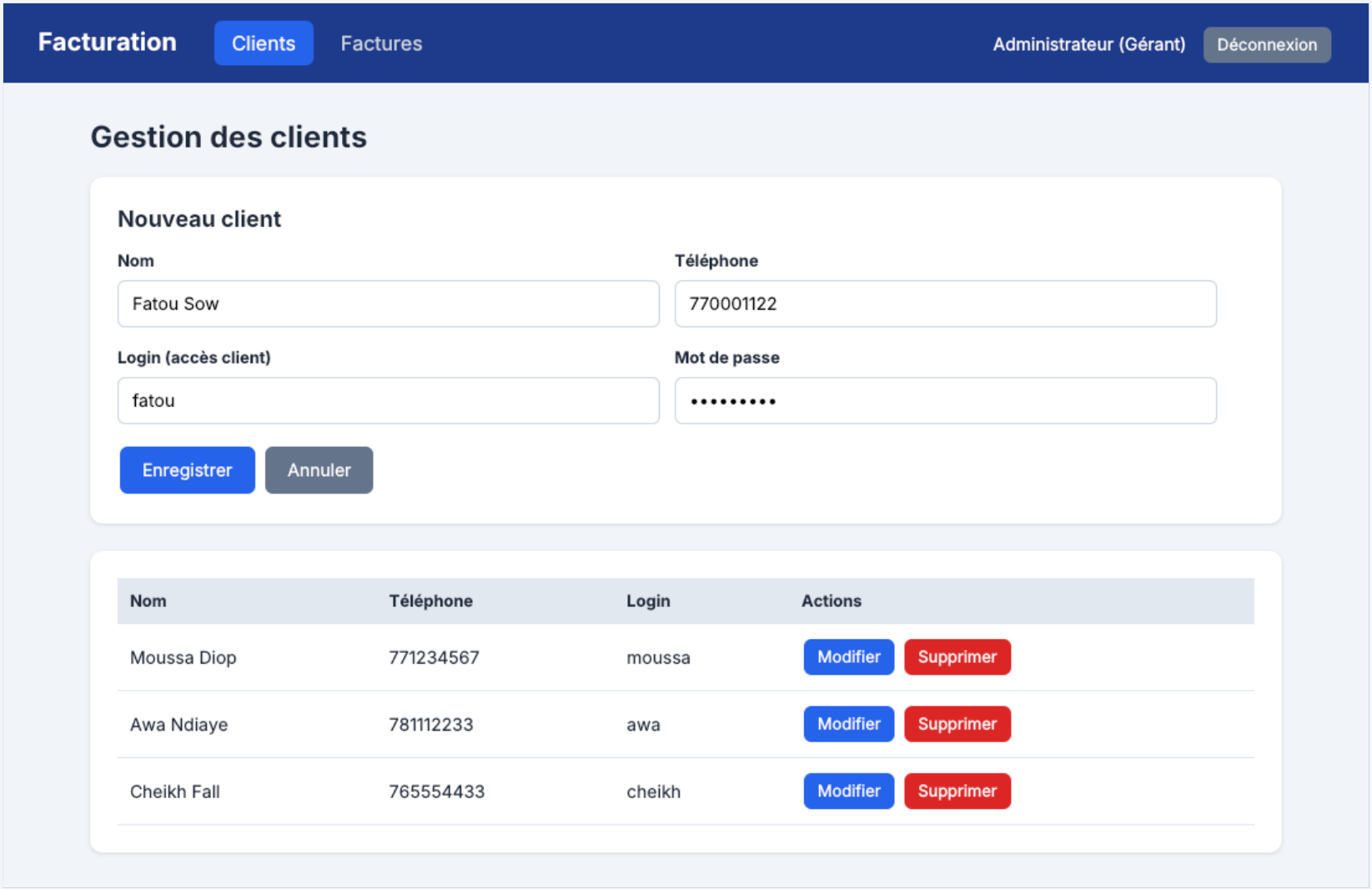
- Ajouter, modifier, supprimer, rechercher (nom ou téléphone)



Liste des clients avec recherche et actions

Espace gérant : ajout d'un client

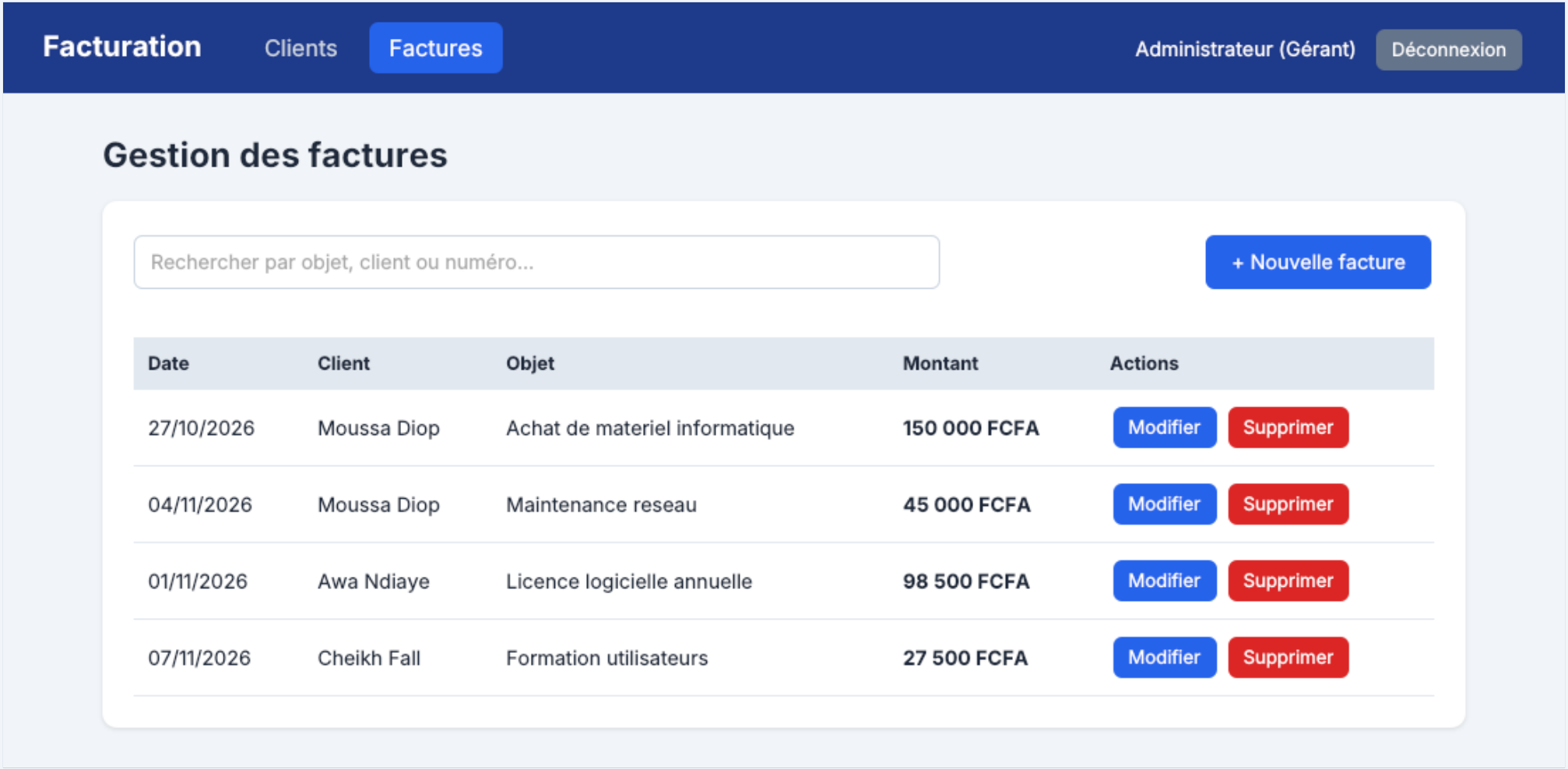
- Login et mot de passe créés ici donnent accès à l'espace client



Formulaire d'ajout / modification d'un client

Espace gérant : factures

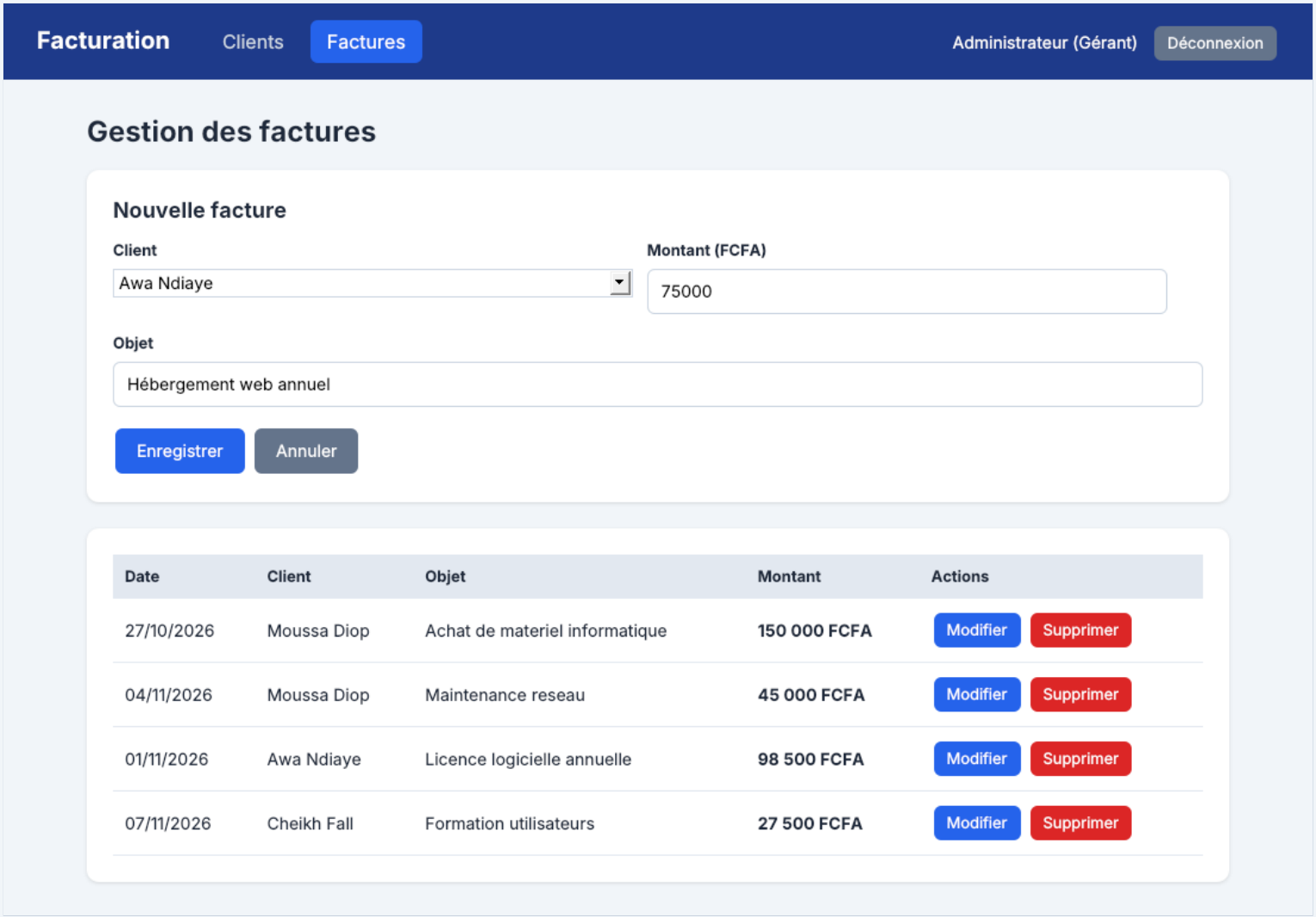
- Recherche par objet, nom du client ou numéro



Liste des factures avec recherche

Espace gérant : ajout d'une facture

- Validation côté serveur : montant positif, client existant



Formulaire de facture : choix du client, montant, objet

Espace client : mes factures

- Accès en lecture seule, protégé par le rôle CLIENT



Le client voit uniquement ses factures et leur total

Conclusion

- Application conforme au sujet du TP2 : React + MongoDB + Spring Boot
- Authentification gérant et client, CRUD et recherche complets
- Sécurité : JWT, BCrypt, contrôle d'accès par rôle, suppression en cascade
- **Perspectives :** pagination, export PDF des factures, statut payé / impayé, tests automatisés, Docker

Merci de votre attention